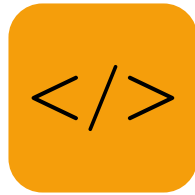


International Institute of Information Technology, Bangalore

Software Production Engineering — Final Project Report



WeCode

Online Coding Judge Platform

A Production-Grade Microservices System

G V Siva Somanath
MT2025048

Nitheesh Vemula
MT2025079

Platform:	LeetCode-style Online Coding Judge
Architecture:	Microservices (6 Spring Boot services)
Stack:	Java 17, Spring Boot 3.2, MySQL, RabbitMQ
DevOps:	Docker, Kubernetes (minikube), Jenkins, Ansible
Observability:	ELK Stack (Elasticsearch, Logstash, Kibana)
Frontend:	React + Vite + Monaco Editor
Auth:	Google OAuth 2.0 + JWT (RBAC)
GitHub:	github.com/sivasomanath2502/OnlineCoding_Judge ↗

Abstract

WeCode is a production-grade online coding judge platform built from scratch, modeled after competitive programming platforms such as LeetCode and HackerRank. The system allows users to solve algorithmic problems, submit code in multiple programming languages (C++, Python, Java), and receive real-time verdicts on their solutions.

The platform is built entirely on a microservices architecture with six independently deployable Spring Boot services, coordinated through a message queue (RabbitMQ) and an API Gateway. Code execution happens inside isolated Docker containers using sandbox security flags to prevent malicious code from affecting the host system. Real-time verdict delivery is implemented using Server-Sent Events (SSE).

The DevOps pipeline automates the entire path from a `git push` to a live Kubernetes deployment — detecting which services changed, building Docker images in parallel, pushing them to DockerHub, and rolling them out to a Kubernetes cluster using Ansible playbooks and `kubect1`. Centralized logging via the ELK stack provides full observability across all services.

This report documents the full journey of building this system — the architectural decisions, the problems encountered, the debugging battles fought, and the reasoning behind every major choice.

DockerHub:	<code>sivasomanath2502</code> (6 public images)
Services:	<code>submission-service</code> , <code>worker-service</code> , <code>result-service</code> , <code>api-gateway</code> , <code>admin-service</code> , <code>auth-service</code>
Verdict types:	<code>ACCEPTED</code> , <code>WRONG_ANSWER</code> , <code>COMPILATION_ERROR</code> , <code>RUNTIME_ERROR</code> , <code>TIME_LIMIT_EXCEEDED</code>

Individual Contributions

G V Siva Somanath (MT2025048)

- **Microservices:** auth-service, result-service, admin-service
- **Kubernetes Deployment:** Authored all Kubernetes manifests including Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims, and the HorizontalPodAutoscaler for worker-service. Managed namespace setup, pod health probes (startup, readiness, liveness), and rolling update strategy.
- **Ansible Automation:** Designed and implemented all Ansible playbooks for deployment orchestration — including pre-deploy setup, per-service image tag resolution, rolling update execution, health checks, and automatic rollback on failure.
- **ELK Stack Integration:** Set up Elasticsearch, Logstash, and Kibana in the Kubernetes cluster. Configured Logstash pipelines and MDC-based distributed tracing across services.

Nitheesh Vemula (MT2025079)

- **Microservices:** submission-service, worker-service, api-gateway
- **Dockerization:** Wrote all Dockerfiles for backend services and the frontend. Implemented the multi-stage frontend build. Configured the Docker sibling pattern for sandboxed code execution. Pre-pulled execution images and tuned container security flags.
- **Jenkins Pipeline Architecture:** Designed the split-pipeline CI/CD system — the root orchestrator pipeline and individual service pipelines. Implemented change detection via `git diff`, parallel service pipeline triggering, build/skip decision logic, versioned image tagging, and the ImagePullBackOff fix using conditional `:latest` fallback.
- **RabbitMQ Integration:** Implemented the async job queue, manual acknowledgement strategy, dead-letter queue configuration, and the `afterCommit` publish pattern.

Contents

Abstract	1
Individual Contributions	2
1 Introduction	5
1.1 Problem Statement	5
1.2 What We Built	5
1.3 Scope and Constraints	5
2 System Architecture	6
2.1 Architecture Overview	6
2.2 Services and Their Roles	7
2.3 The Async Submission Pipeline	8
2.4 API Gateway — More Than a Router	9
2.5 Code Execution — Docker Sibling Pattern	10
3 Key Design Decisions	10
3.1 Shared Database vs. Database-per-Service	10
3.1.1 The Decision	10
3.1.2 Why Not Separate Databases?	10
3.1.3 The Trade-off Acknowledged	11
3.2 UUID Storage as VARCHAR(36)	11
3.2.1 The Bug That Took Longest to Debug	11
3.2.2 The Symptom	11
3.2.3 The Root Cause	11
3.2.4 The Fix	11
3.3 RabbitMQ Message Acknowledgement Strategy	12
3.4 SSE vs WebSocket for Real-Time Verdicts	12
3.5 Pipeline Architecture — Why Split Jenkinsfiles?	13
3.5.1 Initial Approach: Single Jenkinsfile	13
3.5.2 The Split Pipeline Design	13
3.5.3 The ImagePullBackOff Problem	13
4 Technology Stack	13
4.1 Backend	13
4.2 Frontend	14
4.3 DevOps and Infrastructure	14
5 Data Model	15
5.1 Schema Design	15
5.2 Relationships	16
6 Code Execution Engine	16

6.1	Multi-Language Execution	16
6.2	Source File Naming	16
6.3	Sandbox Security Flags	17
6.4	Test Case Execution and Verdict Determination	18
7	CI/CD Pipeline	18
7.1	Pipeline Architecture	18
7.2	Root Pipeline Stages	20
7.3	Service Pipeline Decision Logic	20
7.4	Docker Build and Push	20
7.5	Ansible Deployment	21
8	Kubernetes Deployment	22
8.1	Cluster Architecture	22
8.2	Pod Lifecycle and Health Probes	22
8.3	Service Exposure	23
8.4	Horizontal Pod Autoscaler	23
8.5	Verified Deployment	24
9	ELK Stack and Observability	24
9.1	Why Centralized Logging	24
9.2	Architecture	25
9.3	MDC for Distributed Tracing	25
9.4	Kibana Log Queries	26
9.5	Live View of Histograms	26
10	Authentication and Authorization	27
10.1	Google OAuth 2.0 Flow	27
10.2	RBAC Design	28
10.3	JWT Architecture	29
10.4	Frontend Auth State	29
11	Frontend	29
11.1	Architecture	29
11.2	Key Pages	29
11.3	Monaco Editor Integration	31
11.4	Dark and Light Mode	31
11.5	Dockerization and Environment Variables	32
12	API Reference	32
12.1	All Endpoints Through Gateway (Port 8090)	32
12.2	Submission Request and Response	33
13	Testing the System	34
13.1	Seed Data — Problems and Languages	34
13.2	Verdict Verification	34
13.3	Rate Limiting Test	35
13.4	Verifying Docker Execution	35
14	Deployment Guide	35
14.1	First-Time Setup Sequence	35
14.2	Environment Profiles	36
14.3	Port Forwarding	36

Chapter 1

Introduction

1.1 Problem Statement

Competitive programming platforms like LeetCode, HackerRank, and Codeforces accept code submissions from millions of users and return verdicts within seconds. Building such a system from scratch touches nearly every area of backend engineering — distributed systems, message queues, container isolation, real-time communication, CI/CD pipelines, and observability.

The goal of this project was not to replicate these platforms feature-for-feature, but to build a complete, functioning system that demonstrates every major component of modern backend engineering in one cohesive project. The target was a system that:

- Accepts code in multiple languages and executes it safely
- Returns verdicts in real time without polling
- Deploys automatically on every git push
- Scales worker nodes under load
- Provides full observability via centralized logging
- Enforces role-based access control via OAuth

1.2 What We Built

WeCode is a fully functional online coding judge with the following user-facing features:

- **Problem browser** — lists all problems with difficulty filters and search
- **Code editor** — Monaco editor (same as VS Code) with language switching
- **Run** — compile and execute against custom input, see output immediately
- **Submit** — run against all test cases, receive real-time verdict via SSE
- **Admin panel** — create problems, add/delete test cases, protected behind Google OAuth
- **Role-based access** — `ROLE_ADMIN` for the designated admin email, `ROLE_USER` for everyone else

1.3 Scope and Constraints

The project runs on a local machine using Minikube as the Kubernetes cluster. Jenkins runs as a local CI/CD server. All services are deployed as Docker images pushed to

DockerHub and pulled into the Kubernetes cluster by Ansible playbooks.

One deliberate constraint was the use of a *shared MySQL database* for all services — a pragmatic trade-off explained in detail in Chapter 3. In a production environment, each service would own its own database instance.

Chapter 2

System Architecture

2.1 Architecture Overview

WeCode follows a microservices architecture where each service owns a specific domain of the system. The services communicate through two mechanisms: synchronous HTTP calls for queries that need an immediate response, and asynchronous message passing via RabbitMQ for the core code execution pipeline.

The key insight driving the architecture is that code execution is inherently slow — it can take anywhere from 500ms to 10 seconds depending on the problem, language, and test case count. If the submission endpoint blocked until execution completed, the system would be unusable under any load. The async pipeline solves this by decoupling submission acceptance from execution, returning a submission ID immediately and letting the client receive the verdict via SSE when it is ready.

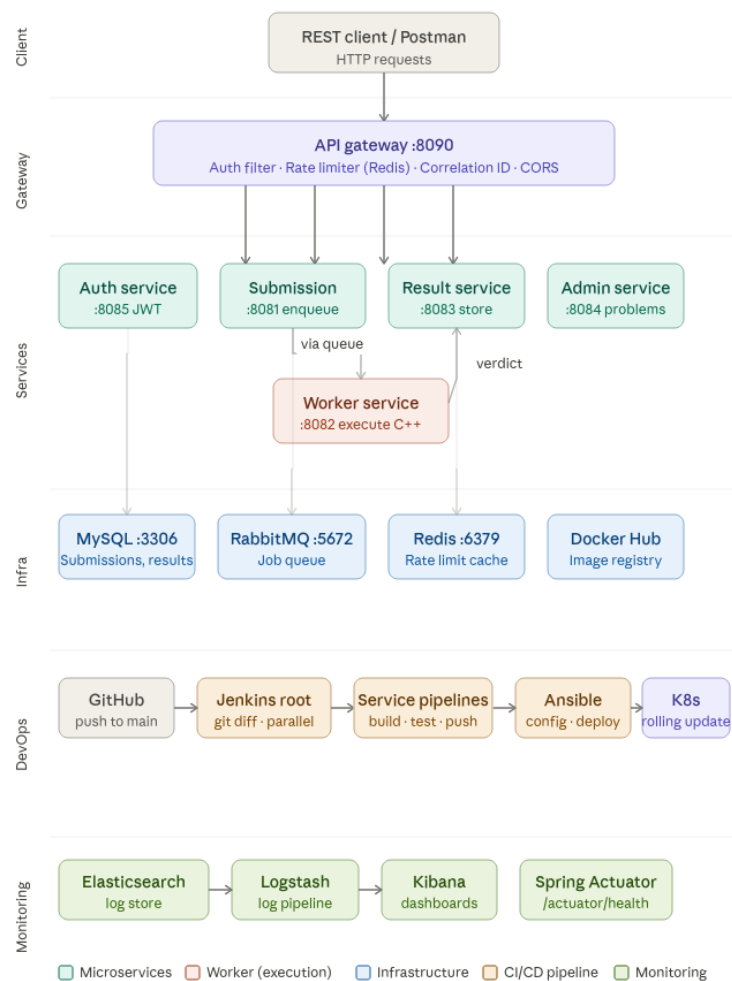


Figure 2.1: High-Level System Architecture — WeCode

2.2 Services and Their Roles

Service	Port	Responsibility
api-gateway	8090	Single entry point. Routes all requests. JWT validation. Rate limiting via Redis. Correlation ID injection.
submission-service	8081	Accepts POST /submissions. Validates problem and language exist. Saves submission to DB. Publishes job to RabbitMQ after DB commit.

Service	Port	Responsibility
worker-service	8082	Consumes jobs from RabbitMQ. Writes code to temp directory. Runs compile and execute steps in Docker containers. Saves verdict to DB. Notifies result-service.
result-service	8083	Serves GET /results/{id}. Manages SSE connections. Receives notify from worker and pushes verdict to waiting clients.
admin-service	8084	CRUD for problems and test cases. Protected by ROLE_ADMIN check at gateway level.
auth-service	8085	Google OAuth flow. Issues JWT tokens. Up-serts users in DB. Assigns ROLE_ADMIN to the designated admin email, ROLE_USER to all others.

Table 2.1: WeCode Microservices — Ports and Responsibilities

2.3 The Async Submission Pipeline

The core of the system is the async execution pipeline. The following is the exact flow from a user clicking Submit to seeing a verdict:

1. User submits code via the frontend — POST /submissions through api-gateway
2. submission-service validates the request (problem exists, language exists)
3. submission-service saves the submission to MySQL with status PENDING
4. After the DB transaction commits, submission-service publishes a job message to RabbitMQ
5. The client opens an SSE stream: GET /results/{id}/stream
6. worker-service consumes the job from the queue (manual acknowledgement)
7. worker-service writes the source file to a temp directory shared with the host
8. worker-service runs `docker run gcc:latest` to compile (for C++/Java)
9. worker-service runs `docker run` for each test case, feeding input via stdin
10. worker-service compares actual output with expected output and assigns a verdict
11. worker-service saves the result to MySQL and updates submission status to COMPLETED
12. worker-service calls POST /internal/notify/{submissionId} on result-service
13. result-service looks up the SSE emitter for that submission ID and pushes the verdict
14. Frontend receives the SSE event and displays the verdict

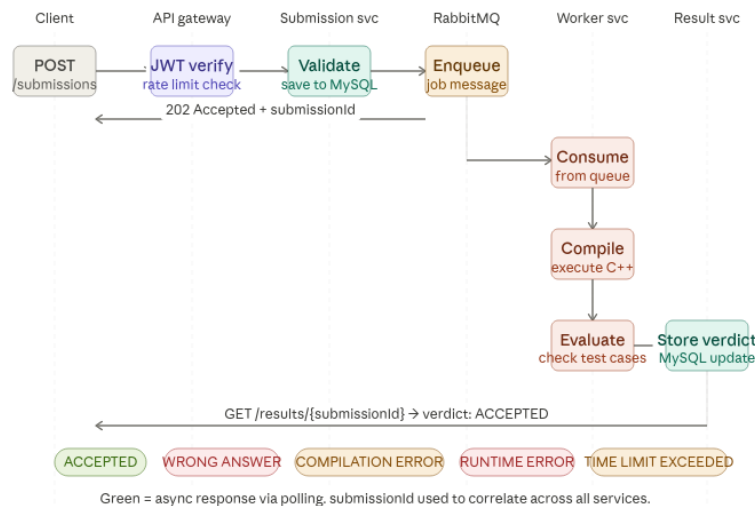


Figure 2.2: Async Submission Pipeline — From POST to Verdict

Why publish after commit, not inside the transaction?

An early version of the system published the RabbitMQ message inside the DB transaction. This caused a race condition: the worker would receive the job, try to read the submission from MySQL, and find nothing because the transaction had not committed yet. The fix was `TransactionSynchronizationManager.registerSynchronization()` with an `afterCommit()` callback. The message is only published after the DB write is durable.

2.4 API Gateway — More Than a Router

The gateway is built on Spring Cloud Gateway (reactive/WebFlux). It does more than route:

- **JWT validation** — every request except `/auth/**` and `/actuator/**` requires a valid JWT. The gateway validates the signature using the same secret as auth-service, without calling auth-service on every request (stateless validation).
- **RBAC** — paths under `/admin/**` require `ROLE_ADMIN`. All other protected paths accept any valid JWT.
- **Rate limiting** — Redis-backed rate limiter using Spring Cloud Gateway's built-in `RedisRateLimiter`. Keyed by `X-User-Id` header or IP fallback. Returns 429 after burst limit.
- **Header injection** — on valid JWT, the gateway injects `X-User-Id`, `X-User-Email`, and `X-User-Role` headers before forwarding downstream. Services read these instead of parsing the JWT themselves.
- **Correlation ID** — every request gets a UUID injected as `X-Correlation-Id`. This ID appears in every service's logs, enabling distributed tracing through Kibana.

2.5 Code Execution — Docker Sibling Pattern

Code execution uses what is called the Docker sibling pattern. The worker-service runs inside a Docker container itself. When it needs to run user code, it calls the Docker daemon on the host machine (via the mounted socket `/var/run/docker.sock`) to spawn a new container. This spawned container is a sibling — it runs alongside the worker container, not inside it.

The file sharing challenge this creates is subtle: the worker writes code to `/judge-tmp/judge-{id}/sol` inside its container. This path is mounted from `/tmp/judge/` on the host. When the worker calls `docker run -v /tmp/judge/judge-{id}:/code gcc:latest`, Docker uses the *host* path — so the gcc container can read the file.

Why not Docker-in-Docker (DinD)?

True DinD runs a Docker daemon inside a container. It requires privileged mode, creates networking complications, and the inner daemon does not share the host's image cache — meaning `gcc:latest` would need to be pulled fresh for every execution container startup. The sibling pattern shares the host's image cache, so pre-pulled images are available immediately with zero pull time.

Chapter 3

Key Design Decisions

3.1 Shared Database vs. Database-per-Service

3.1.1 The Decision

All six services connect to a single MySQL instance with a single `online_judge` database. Each service only reads and writes its own tables. This is a shared database, not a shared schema — services never touch each other's tables directly.

3.1.2 Why Not Separate Databases?

The theoretically correct microservices approach is one database per service. This provides complete data isolation, independent schema evolution, and independent scaling. However, in this project, worker-service needs to read submissions written by submission-service. With separate databases, this cross-service data access would require an API call, adding network latency on the hot path. The shared database lets worker-service read directly — zero network hops on the critical path.

3.1.3 The Trade-off Acknowledged

This is a known compromise. In production at scale, the correct approach would be separate databases with an event-driven data synchronization pattern. worker-service would receive all necessary data in the job message itself, eliminating the need to read from the submissions table at all.

3.2 UUID Storage as VARCHAR(36)

3.2.1 The Bug That Took Longest to Debug

Every entity in every service uses UUID primary keys. The initial implementation used Java's `UUID` type with JPA. In Hibernate 6 (which ships with Spring Boot 3.2), UUID fields are stored and queried as binary bytes (`BINARY(16)`) by default — not as the human-readable hyphenated string representation.

3.2.2 The Symptom

Foreign key constraint violations on every submission. The logs showed:

```
1 Cannot add or update a child row: a foreign key constraint
  fails
2 ('online_judge`.`submissions`, CONSTRAINT `
  FKj5kbdqokftgx992cx24x3s583 `
3 FOREIGN KEY (`problem_id`) REFERENCES `problems` (`id`))
```

3.2.3 The Root Cause

The `Problem` entity used `private UUID id` (Java `UUID` type). The `Submission` entity used `private String id` and had a `@ManyToOne` to `Problem`. When Hibernate wrote the `problem_id` FK into `submissions`, it read the type from the referenced entity and sent binary bytes — even though the column was `VARCHAR(36)`.

3.2.4 The Fix

Change every entity ID in every service to `String` type with `@Column(length=36)` and generate the UUID in `@PrePersist`:

```
1 @Id
2 @Column(name = "id", updatable = false, nullable = false,
  length = 36)
3 private String id;
4
5 @PrePersist
6 protected void onCreate() {
7     if (id == null) {
8         id = java.util.UUID.randomUUID().toString();
9     }
10 }
```

This forces Hibernate to treat every UUID as a plain 36-character string throughout the entire ORM layer.

3.3 RabbitMQ Message Acknowledgement Strategy

Worker-service uses **manual acknowledgement** for all queue messages. With auto-ack, if the worker crashes during execution, the job is lost forever — no verdict, the submission stays PENDING indefinitely.

With manual ack, the message stays in the queue until the worker explicitly calls `channel.basicAck(deliveryTag, false)` after saving the verdict. If the worker crashes before acking, RabbitMQ requeues the message and it is picked up by another worker instance.

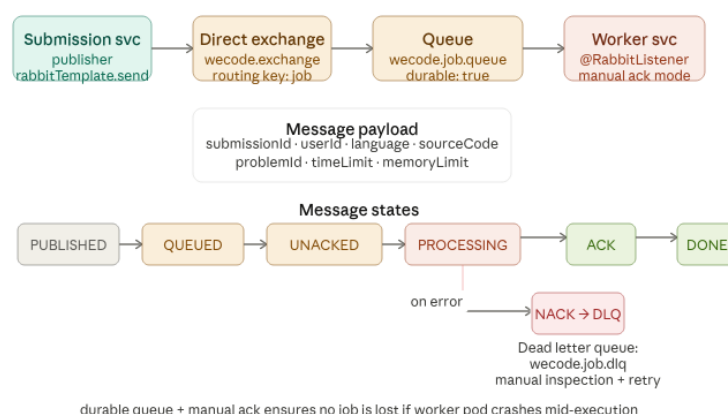


Figure 3.1: RabbitMQ Message Flow and Acknowledgement States

The Dead Letter Queue (`job.dlq`) receives messages that fail repeatedly — after three failed attempts, the message is moved to the DLQ where it can be inspected and replayed manually.

DLQ configuration bug encountered: The initial DLQ configuration used a named dead-letter exchange (`job.dlx`). Messages went to the DLQ immediately without any processing attempts. The fix was to use `"` (the default exchange) as the dead-letter-exchange — the default exchange routes directly by queue name, which is what was actually needed.

3.4 SSE vs WebSocket for Real-Time Verdicts

Server-Sent Events (SSE) was chosen over WebSocket for verdict delivery because:

- **Unidirectional** — verdicts only flow from server to client, never the other direction. SSE is designed for this; WebSocket adds bidirectional complexity that is not needed.
- **HTTP-native** — SSE works over regular HTTP/1.1. It traverses proxies, load balancers, and API gateways without special configuration.
- **Auto-reconnect** — browsers reconnect SSE automatically on network interruption. WebSocket reconnection must be implemented manually.
- **Simpler server** — Spring's `SseEmitter` is a dozen lines of code. WebSocket

configuration in Spring is significantly more involved.

The fallback for environments where SSE fails is polling every 1.5 seconds for up to 30 seconds.

3.5 Pipeline Architecture — Why Split Jenkinsfiles?

3.5.1 Initial Approach: Single Jenkinsfile

The first pipeline was a single Jenkinsfile at the project root that built all five services on every git push. This worked but was wasteful: changing one line in submission-service triggered a full rebuild of all services, each taking 3–4 minutes.

3.5.2 The Split Pipeline Design

The final design uses a root pipeline that *orchestrates* and individual service pipelines that *execute*. The root pipeline runs `git diff` to detect changed files, sets boolean flags per service, and triggers only the changed services in parallel. Each service pipeline determines whether to do a full build or skip, based on the `IS_CHANGED` flag. After all service pipelines complete, the root pipeline calls the Ansible deploy playbook with per-service change flags.

3.5.3 The ImagePullBackOff Problem

A subtle issue appeared when the Ansible deploy tried to pull `submission-service:182` for a service that had not been rebuilt in build 182. That tag did not exist in DockerHub, causing the Kubernetes pod to fail with `ImagePullBackOff`.

The fix is the `service_tags` map in the Ansible playbook: changed services get the new versioned tag, unchanged services get `:latest` — which always exists and is the most recently built image for that service.

Chapter 4

Technology Stack

4.1 Backend

Technology	Version	Role
Java	17	Language for all microservices
Spring Boot	3.2.0	Framework for all microservices
Spring Cloud Gateway	2023.0.0	API Gateway (reactive/WebFlux)

Technology	Version	Role
Spring Data JPA	(Boot-managed)	ORM layer with Hibernate 6
Spring AMQP	3.1.0	RabbitMQ integration
MySQL	8.0	Primary relational database
RabbitMQ	3.12-management	Message broker for async execution
Redis	7-alpine	Rate limiting store for gateway
Lombok	1.18.30	Boilerplate reduction
jjwt	0.12.3	JWT generation and validation
logstash-logback-encoder	7.4	Structured JSON logging to ELK

Table 4.1: Backend Technology Stack

4.2 Frontend

Technology	Version	Role
React	18.2	UI framework
Vite	5.0	Build tool and dev server
Monaco Editor	4.6.0	Code editor (VS Code engine)
React Router	6.20	Client-side routing
Axios	1.6	HTTP client with interceptors

Table 4.2: Frontend Technology Stack

4.3 DevOps and Infrastructure

Technology	Version	Role
Docker	Latest	Containerization of all services
Kubernetes (Minikube)	Latest	Container orchestration
Jenkins	2.x	CI/CD automation server
Ansible	Latest	Deployment automation
Elasticsearch	8.11.0	Log storage and indexing
Logstash	8.11.0	Log processing pipeline
Kibana	8.11.0	Log visualization
DockerHub	—	Container image registry

Table 4.3: DevOps Technology Stack

Chapter 5

Data Model

5.1 Schema Design

The database contains six tables. All UUID primary keys are stored as VARCHAR(36) strings.

Table	Key Columns	Notes
languages	id (INT AUTO_INCREMENT), name, version, compile_cmd, run_cmd, docker_image	Seeds: C++ (gcc:latest), Python (python:3.11-slim), Java (eclipse-temurin:17-alpine)
problems	id (VARCHAR 36), title, description, difficulty ENUM, time_limit_ms	Difficulty: EASY, MEDIUM, HARD
test_cases	id (VARCHAR 36), problem_id FK, input TEXT, expected_output TEXT, order_index	Multiple test cases per problem
submissions	id (VARCHAR 36), problem_id FK, language_id FK, user_id, code TEXT, status ENUM, submitted_at	Status: PENDING, RUNNING, COMPLETED, FAILED
results	id (VARCHAR 36), submission_id FK UNIQUE, verdict ENUM, execution_ms, test_cases_passed, test_cases_total, error_message TEXT	Verdict: ACCEPTED, WRONG_ANSWER, COMPILATION_ERROR, RUNTIME_ERROR, TIME_LIMIT_EXCEEDED
users	id (VARCHAR 36), email UNIQUE, name, picture, role ENUM, google_id UNIQUE, created_at, last_login	Role: ROLE_USER, ROLE_ADMIN

Table 5.1: Database Schema Summary

5.2 Relationships

- problems 1:N test_cases (one problem has many test cases)
- problems 1:N submissions (one problem has many submissions)
- languages 1:N submissions (one language used in many submissions)
- submissions 1:1 results (each submission gets exactly one result)

Chapter 6

Code Execution Engine

6.1 Multi-Language Execution

The worker-service supports C++, Python, and Java. Each language is stored in the database with its compile command, run command, and Docker image. Adding a new language requires only a database insert — no code changes.

Language	Compile Command	Run Command	Docker Image
C++	g++ -o solution solution.cpp	./solution	gcc:latest
Python	(none — interpreted)	python3 solution.py	python:3.11-slim
Java	javac solution.java	java solution	eclipse-temurin:17-alpine

Table 6.1: Supported Languages

Python requires no compilation step — the `compile_cmd` field is empty in the database. The `ExecutionService` checks for a blank `compileCmd` and skips the compile Docker run entirely.

6.2 Source File Naming

The filename matters for some languages. Java requires the public class name to match the filename. The source file is named based on the language:

```
1 private String getSourceFileName(String languageName) {  
2     return switch (languageName.toLowerCase()) {  
3         case "python" -> "solution.py";  
4         case "java"    -> "solution.java";
```

```
5     default      -> "solution.cpp";
6 };
7 }
```

6.3 Sandbox Security Flags

Every execution container (both compile and run steps) runs with hardened security flags:

```
1 docker run \
2     --rm                # Remove container
   after exit
3     --network=none      # No internet
   access
4     --memory=128m       # Memory limit
5     --cpus=0.5          # CPU limit
6     --security-opt=no-new-privileges # Prevent
   privilege escalation
7     --cap-drop=ALL      # Drop ALL Linux
   capabilities
8     --read-only         # Read-only root
   filesystem
9     --user=65534:65534  # Run as nobody:
   nogroup (non-root)
10    --pids-limit=50     # Prevent fork
   bombs
11    --ulimit nofile=64:64 # Limit open file
   descriptors
12    --ulimit nproc=50:50 # Limit process
   count
13    --tmpfs /tmp:rw,noexec,nosuid,size=64m # Writable /tmp
   only
14    -v /tmp/judge/judge_{id}:/code:ro     # Code mounted
   read-only
15    gcc:latest
16    sh -c "g++ -o solution solution.cpp"
```

What each flag prevents:

- `--network=none` — prevents exfiltrating data or downloading payloads
- `--cap-drop=ALL` — removes all Linux capabilities including SETUID, KILL, NET_ADMIN
- `--read-only` — code cannot write to the filesystem except `/tmp`
- `--user=65534:65534` — even if container is compromised, process has no privileges on host
- `--pids-limit=50` — fork bombs cannot spawn unlimited processes
- `noexec` on `/tmp` — cannot execute binaries written to `/tmp`

6.4 Test Case Execution and Verdict Determination

For each test case, the worker writes the input to `input.txt`, runs the execution container, reads stdout and stderr separately, checks if the process finished within the time limit, trims both actual and expected output before comparison, and assigns a verdict. Execution stops at the first failing test case. If all test cases pass, the verdict is `ACCEPTED`.

Chapter 7

CI/CD Pipeline

7.1 Pipeline Architecture

The CI/CD system consists of seven Jenkins pipeline jobs: a root orchestrator (**wecode**) triggered by GitHub webhook on every push, plus individual pipelines for submission-service, worker-service, result-service, api-gateway, admin-service, and wecode-frontend.

Jenkins runs on port 8080 on the local machine. The API Gateway was moved to port 8090 to avoid the conflict.

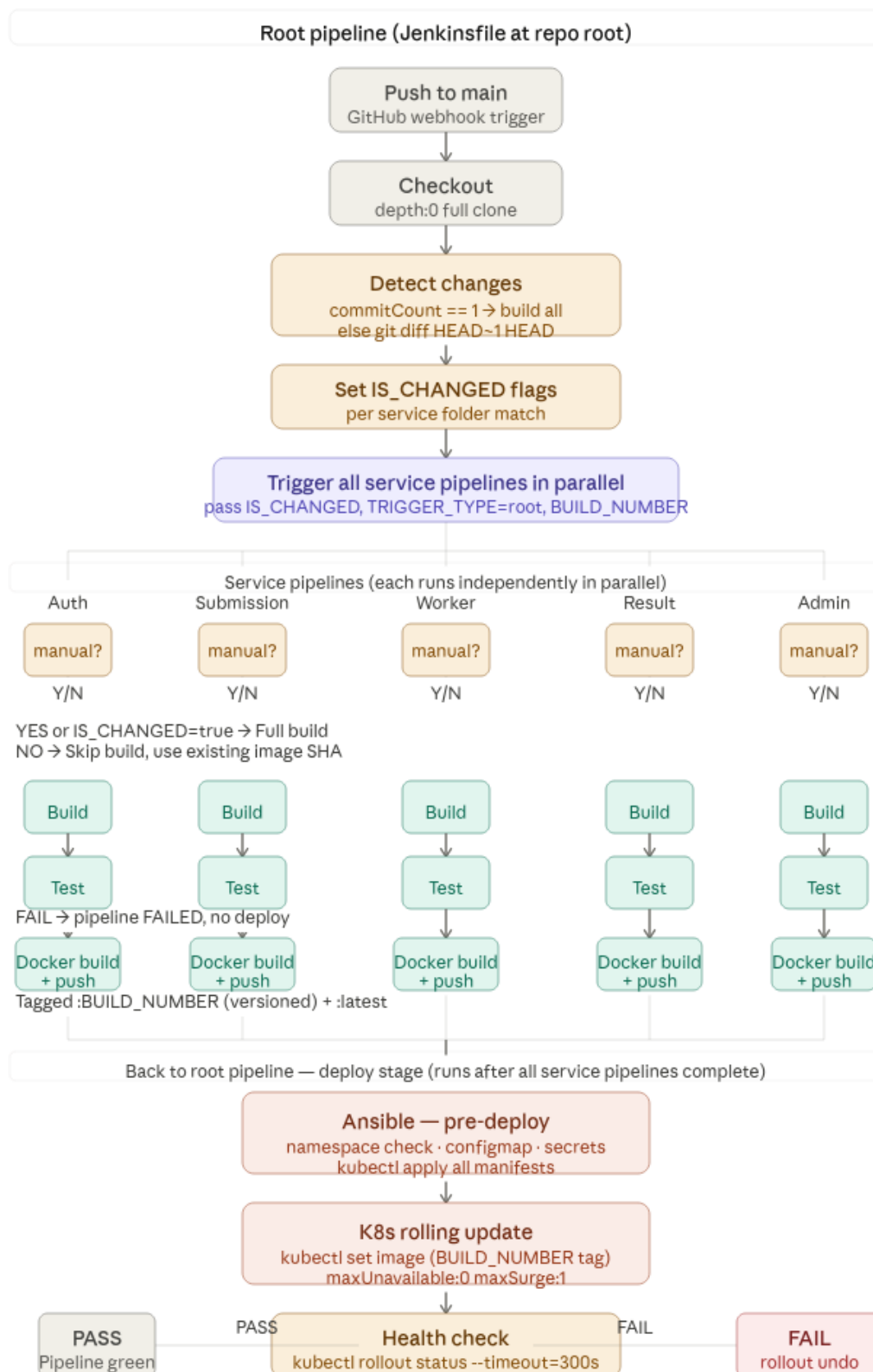


Figure 7.1: Jenkins Pipeline Architecture — Root Orchestration and Parallel Service Pipelines

7.2 Root Pipeline Stages

1. **Checkout** — full depth clone (depth=0) to allow reliable `git diff`
2. **Detect Changes** — `git diff --name-only HEAD~1 HEAD` identifies changed files. Sets `BUILD_*` flags per service. Falls back to building all on first commit.
3. **Trigger Service Pipelines** — triggers all six service pipelines in parallel, passing `IS_CHANGED`, `TRIGGER_TYPE`, and `ROOT_IMAGE_TAG` parameters
4. **Deploy All Services to Kubernetes** — calls Ansible playbook with per-service change flags

7.3 Service Pipeline Decision Logic

Each service pipeline determines whether to do a full build based on how it was triggered:

```
1 stage('Decide Build Path') {
2     steps {
3         script {
4             def isManual = (params.TRIGGER_TYPE == 'manual')
5             def isChanged = params.IS_CHANGED
6             env.DO_FULL_BUILD = (isManual || isChanged) ? '
7         true' : 'false'
8     }
9 }
```

- **Manual trigger** — always full build (developer forcing a redeploy)
- **Root trigger + `IS_CHANGED=true`** — full build (code changed)
- **Root trigger + `IS_CHANGED=false`** — skip build, redeploy with existing image

7.4 Docker Build and Push

Each service builds two tags: the build number (`:182`) and `:latest`:

```
1 docker build \
2     -t sivasomanath2502/submission-service:${IMAGE_TAG} \
3     -t sivasomanath2502/submission-service:latest .
4 docker push sivasomanath2502/submission-service:${IMAGE_TAG}
5 docker push sivasomanath2502/submission-service:latest
```

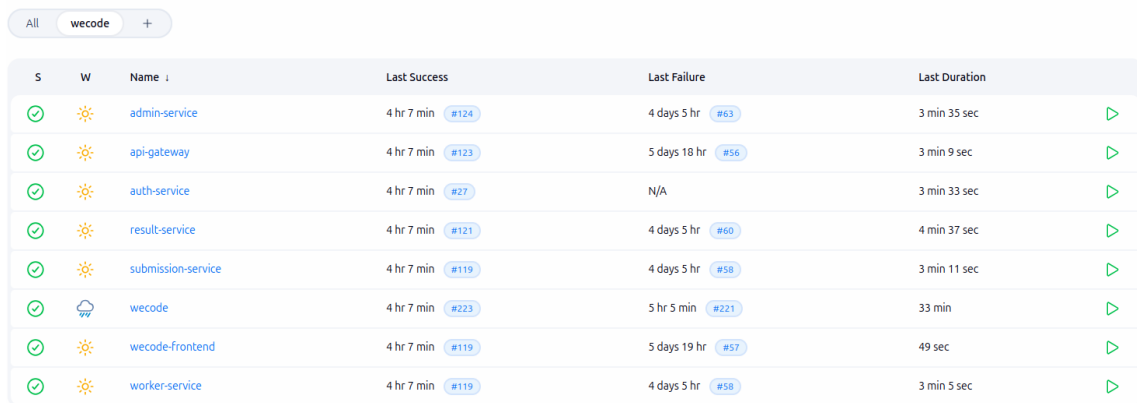
DockerHub stores all versions, enabling rollback by deploying an older tag.

7.5 Ansible Deployment

After all service pipelines complete, the root pipeline calls the Ansible deploy playbook. The playbook resolves per-service image tags:

```
1 service_tags:
2   submission-service: >
3     {{ build_number if changed_submission == 'true' else '
4     latest' }}
5   worker-service: >
6     {{ build_number if changed_worker == 'true' else 'latest'
7     }}
8   }
```

Ansible then iterates through all services sequentially, running `kubectl set image`, `kubectl rollout restart`, and `kubectl rollout status` for each. If a rollout fails, it automatically triggers `kubectl rollout undo` to revert to the previous version.



The Jenkins Dashboard shows a list of service pipelines. All pipelines are in a 'Success' state, indicated by green checkmarks. The table includes columns for status, icon, name, last success, last failure, and last duration.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	admin-service	4 hr 7 min #124	4 days 5 hr #63	3 min 35 sec
✓	☀	api-gateway	4 hr 7 min #123	5 days 18 hr #56	3 min 9 sec
✓	☀	auth-service	4 hr 7 min #27	N/A	3 min 33 sec
✓	☀	result-service	4 hr 7 min #121	4 days 5 hr #60	4 min 37 sec
✓	☀	submission-service	4 hr 7 min #119	4 days 5 hr #58	3 min 11 sec
✓	☁	wecode	4 hr 7 min #223	5 hr 5 min #221	33 min
✓	☀	wecode-frontend	4 hr 7 min #119	5 days 19 hr #57	49 sec
✓	☀	worker-service	4 hr 7 min #119	4 days 5 hr #58	3 min 5 sec

Figure 7.2: Jenkins Dashboard — All Service Pipelines Passing

Stage View



The Stage View for Build #223 shows the timing for each stage. The table includes columns for stage names and their durations. The 'Deploy All Services to Kubernetes' stage is marked as failed.

	Declarative: Checkout SCM	Checkout	Detect Changes	Trigger Service Pipelines	Deploy All Services to Kubernetes	Declarative: Post Actions
Average stage times: (full run time: ~24min 32s)	1s	1s	632ms	1min 1s	15min 39s	309ms
#223 May 14 13:14 1 commit	1s	1s	707ms	4min 46s	28min 57s	324ms
#222 May 14 12:17 No Changes	1s	1s	611ms	16s	17min 53s	304ms
#221 May 14 12:17 1 commit	1s	861ms	632ms	14s	2s failed	297ms
#220 May 14 11:48 No Changes	872ms	897ms	608ms	17s	20min 48s failed	316ms

Figure 7.3: Jenkins Stage View — Build #223 with Stage Timings

Chapter 8

Kubernetes Deployment

8.1 Cluster Architecture

Everything runs in a single `wecode` namespace in a local Minikube cluster. The namespace contains 6 application service Deployments, MySQL with PersistentVolumeClaim (2Gi), RabbitMQ, Redis, Elasticsearch, Logstash, Kibana, 1 HPA for worker-service, and ConfigMap, Secret, AuthSecret objects.

8.2 Pod Lifecycle and Health Probes

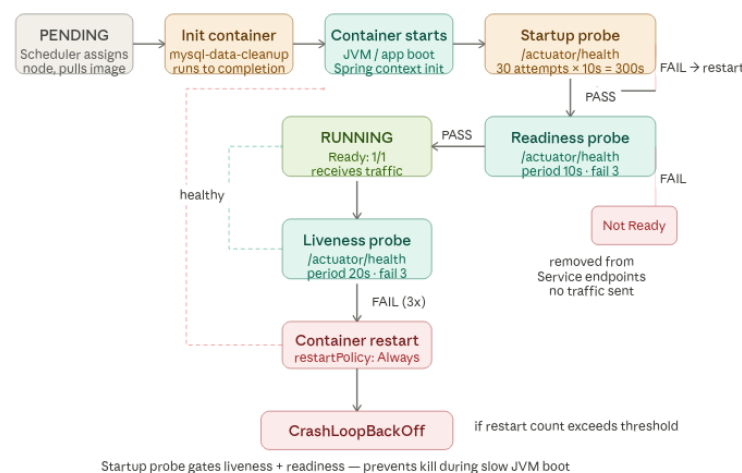


Figure 8.1: Kubernetes Pod Lifecycle — Startup, Readiness, and Liveness Probes

All Spring Boot services expose `/actuator/health`. Three probes are configured:

- **Startup probe** — 30 attempts at 10s intervals (300s total). Prevents liveness probe from killing the container during slow JVM boot.
- **Readiness probe** — period 10s, fail 3. Removes the pod from Service endpoints if it fails, so no traffic is sent.
- **Liveness probe** — period 20s, fail 3. Restarts the container if it fails. `CrashLoopBackOff` occurs if restart count exceeds the threshold.

8.3 Service Exposure

Service	Type	NodePort	Notes
api-gateway	NodePort	30090	External access point
wecode-frontend	NodePort	30000	Frontend UI
rabbitmq	NodePort	31672	Management UI access
kibana	NodePort	31601	Log visualization
submission-service	ClusterIP	—	Internal only
worker-service	ClusterIP	—	Internal only
result-service	ClusterIP	—	Internal only
admin-service	ClusterIP	—	Internal only
auth-service	ClusterIP	—	Internal only
mysql	ClusterIP	—	Internal only
redis	ClusterIP	—	Internal only
elasticsearch	ClusterIP	—	Internal only
logstash	ClusterIP	—	Internal only

Table 8.1: Kubernetes Service Exposure

8.4 Horizontal Pod Autoscaler

The worker-service has an HPA configured for automatic scaling under load:

```

1 spec:
2   scaleTargetRef:
3     name: worker-service
4   minReplicas: 2
5   maxReplicas: 10
6   metrics:
7     - type: Resource
8       resource:
9         name: cpu
10        target:
11          type: Utilization
12          averageUtilization: 60
13     - type: Resource
14       resource:
15         name: memory
16        target:
17          type: Utilization
18          averageUtilization: 70
19   behavior:
20     scaleUp:
21       stabilizationWindowSeconds: 30
22     scaleDown:

```

The worker is the natural bottleneck — it does CPU-heavy Docker execution. With HPA, the cluster scales from 2 to 10 worker replicas automatically as submission load increases. RabbitMQ distributes messages across all active workers.

8.5 Verified Deployment

"NAME	READY	STATUS	RESTARTS	AGE",
"admin-service-7f79f44d5c-bg5xh	1/1	Running	0	12m",
"admin-service-7f79f44d5c-dkkj2	1/1	Running	0	10m",
"api-gateway-6f456f786-fcjqx	1/1	Running	0	9m11s",
"api-gateway-6f456f786-jdbqx	1/1	Running	0	8m",
"auth-service-756bf7cc58-mnvwf	1/1	Running	0	4m21s",
"auth-service-756bf7cc58-zzv2z	1/1	Running	0	6m54s",
"elasticsearch-549c94bbc-bt6zz	1/1	Running	0	27m",
"kibana-5566d59dd5-ttlq7	1/1	Running	1 (24m ago)	27m",
"logstash-594d886dd7-jmmvb	1/1	Running	0	27m",
"mysql-969cf9c6c-8r76z	1/1	Running	0	28m",
"rabbitmq-7d97548cff-b549t	1/1	Running	0	28m",
"redis-5445858885-rn8br	1/1	Running	0	28m",
"result-service-86d78bf598-7srn9	1/1	Running	0	14m",
"result-service-86d78bf598-tdc5z	1/1	Running	0	17m",
"submission-service-86584bf9f8-76vqt	1/1	Running	0	20m",
"submission-service-86584bf9f8-ffqsc	1/1	Running	0	23m",
"wecode-frontend-bdc96c88f-p5mh8	1/1	Running	0	116s",
"worker-service-6bfbf46476-k4bwc	1/1	Terminating	0	23m",
"worker-service-9f5964b66-j6x45	1/1	Running	0	94s",
"worker-service-9f5964b66-twvx2	1/1	Running	0	42s"

Figure 8.2: `kubectl get pods -n wecode` — All Pods Running

Chapter 9

ELK Stack and Observability

9.1 Why Centralized Logging

In a microservices system, a single user action can touch six different services. When something goes wrong, the relevant log entries are scattered across six different containers. Without centralized logging, debugging requires running `kubectl logs` on each pod separately and mentally correlating timestamps.

9.2 Architecture

1. Spring Boot services emit structured JSON logs via LogstashTcpSocketAppender to `logstash:5000`
2. Logstash parses the JSON, adds tags (`verdict_event`, `submission_event`, `error_event`), and writes to Elasticsearch
3. Elasticsearch indexes into daily indices: `wecode-submission-service-2026.05.14`
4. Kibana provides search, filter, and dashboard visualization

The Logstash appender is only active on the `docker` and `kubernetes` Spring profiles. Local development uses console logging only.

9.3 MDC for Distributed Tracing

MDC (Mapped Diagnostic Context) attaches contextual fields to every log line generated during request processing. In worker-service, the `submissionId` is put into MDC at the start of each job:

```

1 MDC.put("submissionId", jobMessage.getSubmissionId());
2 try {
3     // all log.info() calls inside here include submissionId
4     // automatically
5 } finally {
6     MDC.clear();
7 }

```

In Kibana, filtering by `submissionId`: `"abc123"` shows every log line from every service that touched that submission.

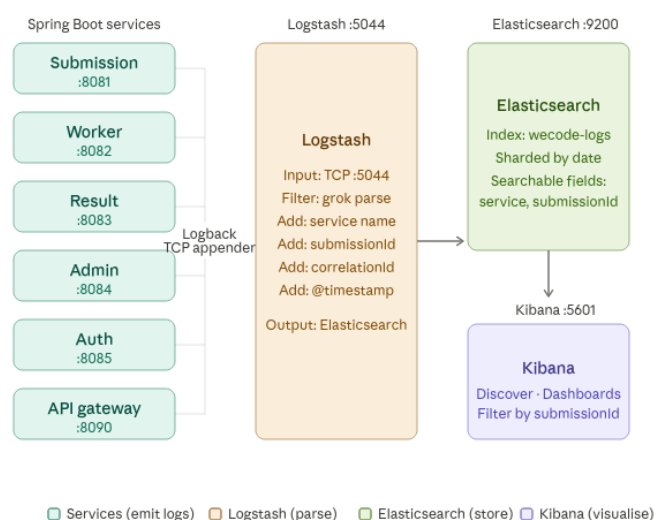


Figure 9.1: MDC-Based Distributed Tracing — `submissionId` Across Services

9.4 Kibana Log Queries

```

1 # All errors
2 log_level: ERROR
3
4 # All lines for a specific submission
5 submissionId: "your-submission-id"
6
7 # All ACCEPTED verdicts
8 log_message: *ACCEPTED*
9
10 # Worker service logs only
11 service_name: worker-service
12
13 # All verdict events tagged by Logstash
14 tags: verdict_event

```

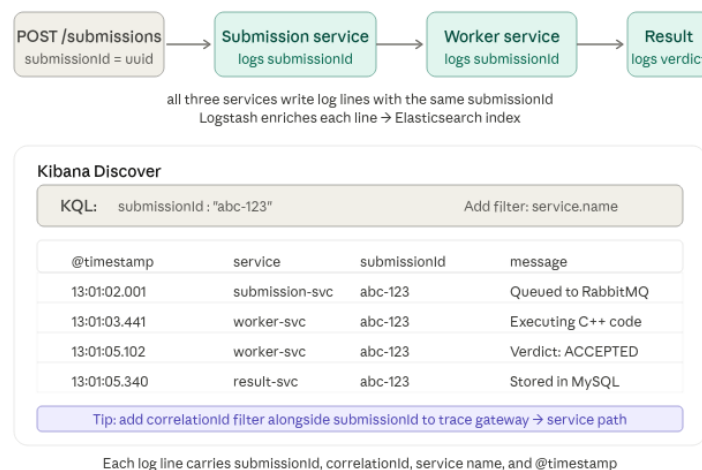
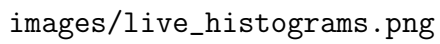


Figure 9.2: Kibana Discover — Filtering Logs by submissionId Across All Services

9.5 Live View of Histograms

Centralized observability is further enhanced by Kibana’s visualization capabilities. Beyond raw logs, the system provides a live view of submission trends, error rates, and verdict distributions through histograms. These visualizations allow for real-time monitoring of the system’s health and performance, identifying spikes in failure rates or unusual submission patterns immediately.

A screenshot of the Kibana dashboard showing submission histograms and metrics. The image is mostly blank, suggesting the content is either missing or rendered as a placeholder.

images/live_histograms.png

Figure 9.3: Kibana Dashboard — Live View of Submission Histograms and Metrics

Chapter 10

Authentication and Authorization

10.1 Google OAuth 2.0 Flow

Authentication uses Google OAuth 2.0, chosen because it eliminates the need to store and manage passwords. Users log in with their existing Google accounts. The system never sees their password.

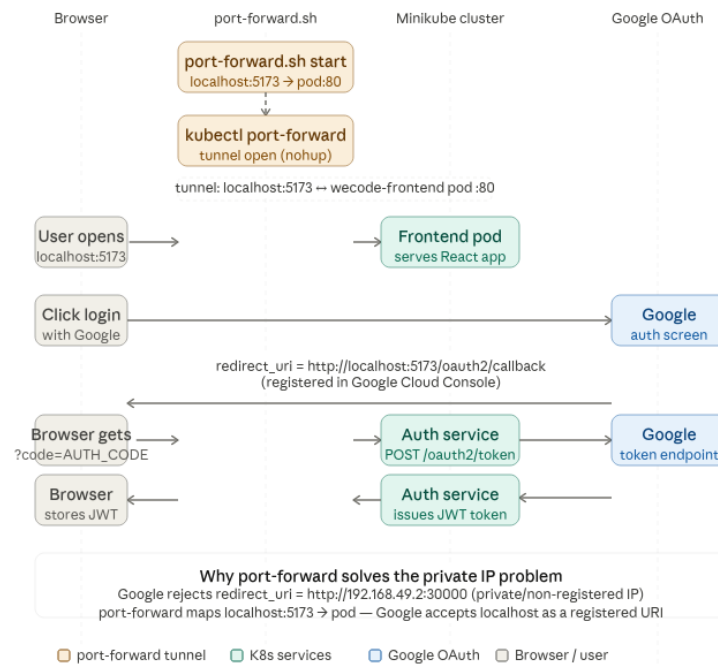


Figure 10.1: Google OAuth 2.0 Flow — From Login Click to JWT Token

The flow: the user clicks Sign In, the frontend redirects to `/auth/google`, auth-service builds a Google authorization URL, the user approves on Google’s consent screen, auth-service exchanges the code for an access token (server-to-server), calls Google’s userinfo API, upserts the user in the database, generates a JWT with role claims, and redirects to the frontend with the token in the URL.

10.2 RBAC Design

The role assignment is entirely determined by the email address registered in `app.admin-email` in auth-service’s properties. There is no admin management UI — the admin is whoever owns that email. This is simple, auditable, and requires no database management.

```

1 Role role = email.equalsIgnoreCase(adminEmail)
2   ? Role.ROLE_ADMIN
3   : Role.ROLE_USER;

```

Path	Access	Enforced By
<code>/auth/**</code>	Public (no token)	JwtAuthFilter whitelist
<code>/actuator/**</code>	Public	JwtAuthFilter whitelist
<code>/admin/**</code>	ROLE_ADMIN only	JwtAuthFilter RBAC check
<code>/submissions</code>	Any valid JWT	JwtAuthFilter
<code>/results/**</code>	Any valid JWT	JwtAuthFilter

Table 10.1: Route Access Control

10.3 JWT Architecture

The JWT is validated at the gateway level — not in individual services. Individual services are never directly exposed to the internet. JWT validation happens once at the entry point. Services receive pre-validated user context via headers (X-User-Id, X-User-Email, X-User-Role). The JWT secret is shared between auth-service (signs) and api-gateway (verifies) via Kubernetes secret.

10.4 Frontend Auth State

Auth state is managed by a React context (`AuthContext`). The token is stored in `sessionStorage` — not `localStorage`. `sessionStorage` is cleared when the browser session ends and is isolated per tab. On every app mount, the context checks `sessionStorage` for a stored token and validates it with `GET /auth/me`. Axios interceptors attach the token to every outgoing request and redirect to `/login` on receiving 401.

Chapter 11

Frontend

11.1 Architecture

The frontend is a React SPA built with Vite. All API calls go through the API Gateway — the frontend never talks directly to any backend service. The gateway URL is configured via the `VITE.GATEWAY.URL` environment variable, baked into the build at compile time.

11.2 Key Pages

Page	Description
<code>ProblemsPage</code>	Lists all problems with search and difficulty filtering (EASY/MEDIUM/HARD). Click a problem to open the editor.

Page	Description
ProblemDetailPage	Two-panel layout: left shows problem description and sample test cases, right has Monaco editor with language switching. Two buttons: Run (compile against custom input) and Submit (run against all test cases). Verdict appears via SSE.
AdminPage	Create problems with test cases. Expand existing problems to see, add, or delete test cases. Delete problems (cascade deletes test cases first to avoid FK violations). Protected — only accessible to <code>ROLE_ADMIN</code> .
LoginPage	WeCode branding with a single “Continue with Google” button. Redirects to Google OAuth.
AuthCallbackPage	Receives token from OAuth redirect URL params, stores in sessionStorage, cleans URL, redirects to home.

Table 11.1: Frontend Pages

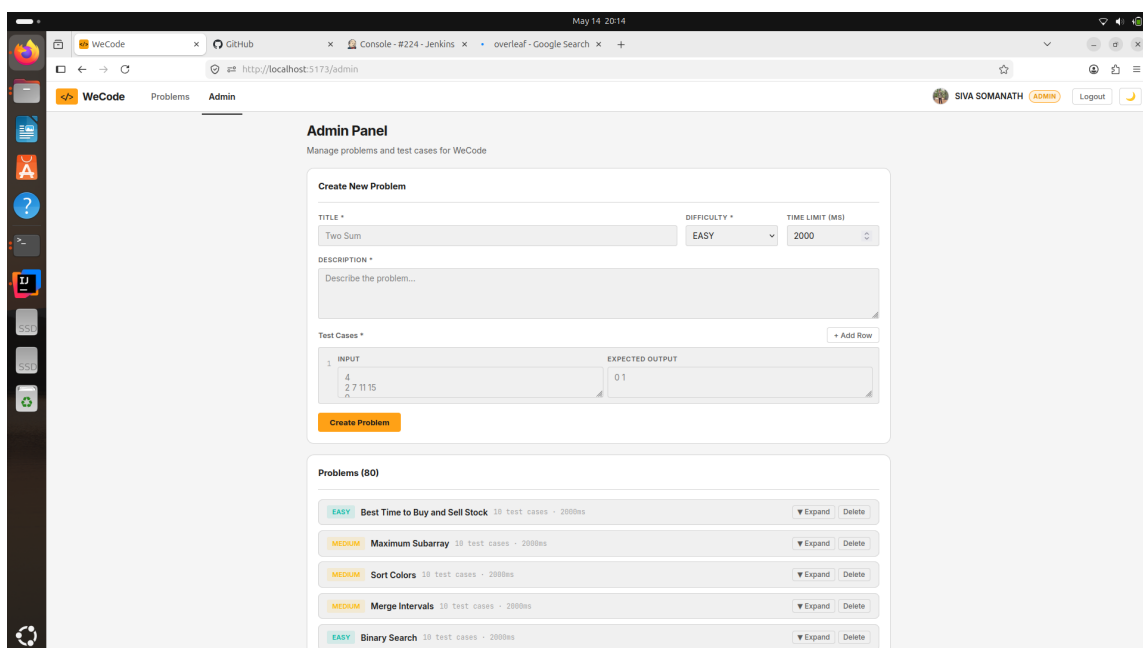


Figure 11.1: WeCode Frontend — Admin Panel for Problem Management

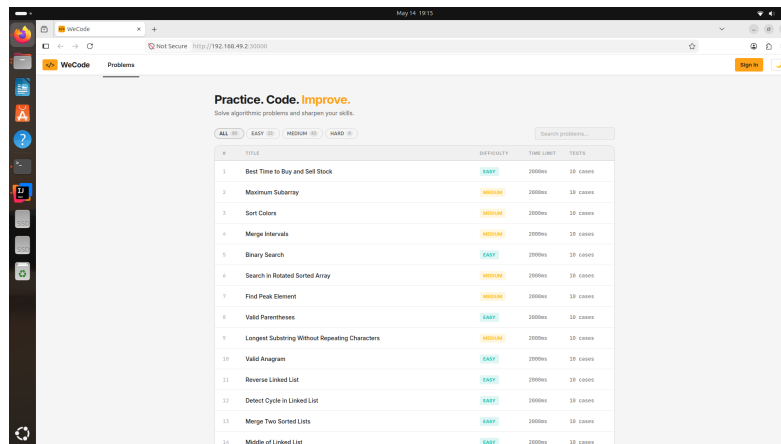


Figure 11.2: WeCode Frontend — Problems Page with Difficulty Filters

11.3 Monaco Editor Integration

The code editor uses Monaco Editor — the same engine that powers VS Code. Each language has a default code template that loads when the user switches languages. The editor supports syntax highlighting, code folding, and tab completion for all three supported languages.

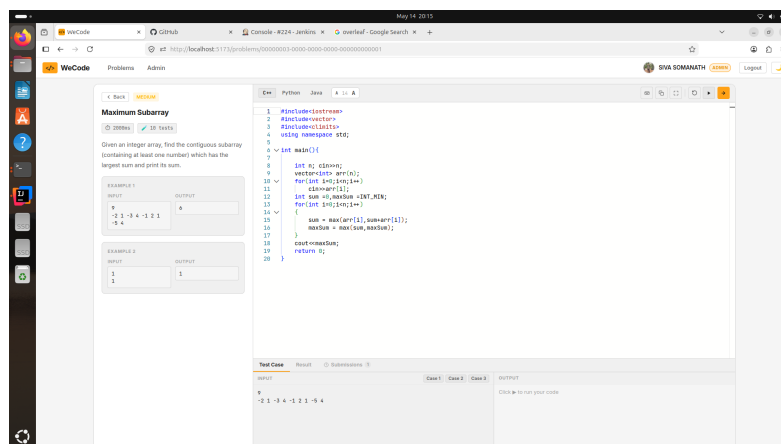


Figure 11.3: WeCode Frontend — Problem Detail Page with Monaco Code Editor

11.4 Dark and Light Mode

The UI supports both dark and light themes, toggled by the moon/sun button in the navbar. Theme preference is persisted in `localStorage`. All colors are CSS variables — switching theme updates the CSS `data-theme` attribute on `document.documentElement`, and all components respond automatically.

11.5 Dockerization and Environment Variables

The frontend is containerized in a multi-stage Dockerfile:

```
1 FROM node:20-alpine AS builder
2 ARG VITE_GATEWAY_URL=http://localhost:8090
3 ENV VITE_GATEWAY_URL=$VITE_GATEWAY_URL
4 RUN npm ci && npm run build
5
6 FROM nginx:alpine
7 COPY --from=builder /app/dist /usr/share/nginx/html
8 COPY nginx.conf /etc/nginx/conf.d/default.conf
```

The `VITE_GATEWAY_URL` is a build-time argument because Vite inlines environment variables into the JavaScript bundle — runtime environment variables have no effect. In Kubernetes, the frontend container is always accessed from the user’s browser, which means the gateway URL must be a browser-accessible address (`localhost:8090` via port-forward), not an internal cluster DNS name.

nginx serves the built React SPA with SPA routing support (`try_files $uri $uri/ /index.html`) so that refreshing on any route works correctly.

Chapter 12

API Reference

12.1 All Endpoints Through Gateway (Port 8090)

Method	Path	Auth quired	Re-	Description
<i>Auth Service</i>				
GET	/auth/google	None		Redirect to Google OAuth
GET	/auth/google/callback	None		Receive OAuth code, issue JWT
GET	/auth/me	JWT		Get current user info
POST	/auth/logout	JWT		Logout (client discards token)
<i>Admin Service — ROLE_ADMIN only</i>				
POST	/admin/problems	ADMIN		Create problem with test cases

Method	Path	Auth required	Re-	Description
GET	/admin/problems	ADMIN		List all problems
GET	/admin/problems/{id}	ADMIN		Get problem with test cases
POST	/admin/problems/{id}/test-cases	ADMIN		Add test cases to problem
DELETE	/admin/problems/{id}	ADMIN		Delete problem
DELETE	/admin/test-cases/{id}	ADMIN		Delete test case
<i>Submission Service</i>				
POST	/submissions	JWT		Submit code for judging
<i>Result Service</i>				
GET	/results/{id}	JWT		Poll for result
GET	/results/{id}/stream	JWT		SSE stream for real-time verdict
POST	/internal/notify/{id}	Internal		Worker notifies of verdict

Table 12.1: WeCode API Reference

12.2 Submission Request and Response

```

1 // POST /submissions
2 {
3     "problemId": "11111111-1111-1111-1111-111111111111",
4     "languageId": 1,
5     "userId": "user-001",
6     "code": "#include<iostream>\nusing namespace std;\n..."
7 }
8
9 // Response 202 Accepted
10 {
11     "submissionId": "a3f4c2d1-...",
12     "status": "PENDING",
13     "message": "Submission accepted for evaluation"
14 }

```

```

1 // GET /results/{submissionId} -- ACCEPTED
2 {
3     "submissionId": "a3f4c2d1-...",
4     "status": "COMPLETED",
5     "verdict": "ACCEPTED",
6     "executionMs": 1340,
7     "testCasesPassed": 3,

```

```

8     "testCasesTotal": 3,
9     "errorMessage": null
10 }
11
12 // GET /results/{submissionId} -- WRONG ANSWER
13 {
14     "submissionId": "...",
15     "status": "COMPLETED",
16     "verdict": "WRONG_ANSWER",
17     "executionMs": 890,
18     "testCasesPassed": 1,
19     "testCasesTotal": 3,
20     "errorMessage": "Expected: 0 1 | Got: 1 0"
21 }

```

Chapter 13

Testing the System

13.1 Seed Data — Problems and Languages

Before any submissions can be tested, seed data must be inserted:

```

1 INSERT INTO languages (name, version, compile_cmd, run_cmd,
2     docker_image)
3 VALUES
4 ('C++', 'g++ 13', 'g++ -o solution solution.cpp',
5     './solution', 'gcc:latest'),
6 ('Python', 'Python 3.11', '',
7     'python3 solution.py', 'python:3.11-slim'),
8 ('Java', 'OpenJDK 17', 'javac solution.java',
9     'java solution', 'eclipse-temurin:17-alpine');

```

Six problems are seeded with fixed UUIDs: Two Sum, Reverse String, FizzBuzz, Find Maximum, Palindrome Check, and Count Vowels.

13.2 Verdict Verification

Verdict	How to Trigger
ACCEPTED	Submit correct C++ Two Sum solution
WRONG_ANSWER	<code>int main(){ cout << "wrong"; }</code>

Verdict	How to Trigger
COMPILATION_ERROR	Submit invalid C++ code
TIME_LIMIT_EXCEEDED	<code>int main(){ while(true){} }</code>
RUNTIME_ERROR	Submit code with null pointer dereference

Table 13.1: Verdict Test Cases

13.3 Rate Limiting Test

```

1 for i in {1..15}; do
2     echo -n "Request $i: "
3     curl -s -o /dev/null -w "%{http_code}\n" \
4         -H "X-User-Id: test-user-123" \
5         http://localhost:8090/admin/problems
6 done
7 # Expected: 200 200 200 ... 429 429 429

```

13.4 Verifying Docker Execution

```

1 # Terminal 1 -- Watch Docker containers in real time
2 watch -n 1 'docker ps --format "table {{.Names}}\t{{.Image}}\t
3     {{.Status}}"'
4
5 # Terminal 2 -- Submit code
6 # A gcc:latest container will appear and disappear for each
7     submission

```

Chapter 14

Deployment Guide

14.1 First-Time Setup Sequence

The order of operations matters critically on first deployment:

1. **Generate JWT secret** — `openssl rand -base64 64` — use this value in both `auth-service` and `api-gateway` properties
2. **Fill secrets** — populate `k8s/auth-secret.yaml` and `application-local.properties` with real Google OAuth credentials and JWT secret

3. **Apply auth secret to Kubernetes manually** — `kubectl apply -f k8s/auth-secret.yaml`. This is a one-time manual step — secrets contain real credentials and must never go through the pipeline.
4. **Build and verify locally** — `mvn clean package -DskipTests` — confirm all JARs build
5. **Push to GitHub** — this triggers Jenkins pipeline automatically via webhook
6. **Watch Jenkins** — all services build in parallel, images push to DockerHub, Ansible deploys to Kubernetes

14.2 Environment Profiles

Each service has three property files:

- `application.properties` — common non-sensitive config (port, app name, logging)
- `application-local.properties` — localhost URLs with placeholder secrets for local development
- `application-docker.properties` — container DNS names, reads secrets from environment variables

Profile is activated by `SPRING_PROFILES_ACTIVE=docker` environment variable in Kubernetes and Docker Compose.

14.3 Port Forwarding

```
1 cd wecode/k8s
2 ./port-forward.sh           # Start all port forwards
3 ./port-forward.sh status   # Check status
4 ./port-forward.sh stop     # Stop all
5
6 # After starting:
7 # http://localhost:5173     Frontend
8 # http://localhost:8090     API Gateway
9 # http://localhost:15672    RabbitMQ Management
10 # http://localhost:5601     Kibana (Live Histograms & Logs)
```

Appendix A

Project Structure

```
1 wecode/
2     Jenkinsfile                                # Root orchestrator
3     pipeline
4         wecode-backend/
5             pom.xml                            # Parent Maven POM
6             api-gateway/
7             submission-service/
8             worker-service/
9             result-service/
10            admin-service/
11            auth-service/
12        wecode-frontend/
13            src/
14                api/client.js
15                contexts/
16                    AuthContext.jsx
17                    ThemeContext.jsx
18                pages/
19                    ProblemsPage.jsx
20                    ProblemDetailPage.jsx
21                    AdminPage.jsx
22                    LoginPage.jsx
23                    AuthCallbackPage.jsx
24                components/
25            Dockerfile
26        ansible/
27            playbooks/
28                deploy-app-k8s.yml
29                deploy-service-k8s.yml
30                tasks/roll-service.yml
31        k8s/
32            namespace.yaml
33            configmap.yaml
34            secret.yaml
35            auth-secret.yaml                    # NOT in git
36            mysql/
37            rabbitmq/
38            redis/
39            submission-service/
40            worker-service/
41            hpa.yaml
```

```
41      result-service/  
42      admin-service/  
43      api-gateway/  
44      auth-service/  
45      frontend/  
46      elk/  
47      apply-all.sh  
48      port-forward.sh
```